Lctr 19: Condition Code Suffixes and Conditional Branch

Jianhua Liu

Spring 2026

==Topics==

Core concepts and applications:

- Condition code suffixes based on the N, Z, C, and V flags
- Comparison instructions
- Branch and conditional branch instructions
- Basic `if-else`, `for`, and `while` constructs in ASM

Demo project:

- The basic `if-else` construct in ASM
- The basic `for` and `while` constructs in ASM

19.1 Introduction

19.1.1 Program flow control in C

In a common C program, we can use a combination of the following constructs to change the flow of the program in a function:

- The `if-else` and `switch` constructs for selection.
- The `for`, `while`, and `do-while` constructs for loop control.

Note that you may have seen the terms of `if-else` statement or `for` loop statement elsewhere. Here, we prefer to call these statements as **constructs** to stress that they include the code contented in the curly braces as well. This inclusion will be useful when discussing ASM programming.

19.1.2 ASM conditional branch instructions and conditional execution

The above C constructs can be implemented in ASM using a combination of the following instructions:

- The branch and conditional branch instructions.
- The conditional execution instructions, which are normal instructions that can be executed based on condition checking.

In this lecture, we focus on the following topics:

- The principle of number comparison based on the NZCV flags.
- The branch and conditional branch instructions as well as conditional execution.

19.2 Demo project

We use a demo project, `fo1s_conditional_branch`, to illustrate how to use ASM conditional branching and conditional executions to implement the basic `if-else`, `for`, and `while` constructs in ASM.

With these basic constructs, we can move forward to cover the load and store instructions with some realistic examples in later lectures.

19.3 Comparison instructions and condition code suffixes

The conditional branch instructions are based on the **condition code suffixes** (CCSs), an extension which can be applied to many instructions. These suffixes are based on the status flags that we have discussed before. These status flags can be used to facilitate many functionalities. Here, we consider how to use these flags for conditional branching; as such, we call these status flags **condition flags**.

19.3.1 A quick review of condition flags

Skipped. See notes of the previous lecture.

19.3.2 Comparison instructions

Before performing the conditional execution instructions, we need to set the above condition flags. This is done by using the comparison instructions, as listed below.

Instruction	Description	Affected flags
`CMP Rn, Op2`	Compare: `Rn - Op2`	N, Z, C, V
`CMN Rn, Op2`	Compare negative: `Rn + Op2`	N, Z, C, V
`TST Rn, Op2`	Test against a mask: `Rn & Op2`	N, Z, C
`TEQ Rn, Op2`	Test equivalence: `Rn ^ Op2`	N, Z, C

We mainly focus on the `CMP` instruction in the sequel.

19.3.3 Seeing the numbers as signed or unsigned

We have to track the data types for comparison cases. Consider the comparison of 8-bit numbers 0x01 and 0xFF:

- For signed numbers, 0x01 is greater than 0xFF, which is -1 in decimal.
- For unsigned numbers, 0x01 is smaller than 0xFF, which is 255 in decimal.

Hence, we have to distinguish between the comparison of signed and unsigned numbers. Note that we must have both numbers in the same type—both signed or unsigned.

19.3.4 Condition code suffixes for signed and unsigned cases

Below is a table for the common condition code suffices we use very often:

C relational test	Signed comparison	Signed CCS	Unsigned comparison	Unsigned CCS
==	Equal to	EQ	Equal to	EQ
!=	Not equal to	NE	Not equal to	NE
>	Greater than	GT	Higher than	HI
>=	Greater than or equal to	GE	Higher than or the same	HS
<=	Less than or equal to	LE	Lower than or the same	LS
<	Less than	LT	Lower than	LO

19.3.5 Relationship testing and condition flags for signed integers

Consider `CMP R0, R1` for signed integers in `R0` and `R1`:

- `V = 0`. No overflow, implying the comparison result is correct. Two situations:
  - `N = 0`. Result is non negative, meaning R0 - R1 >= 0, leading to R0 >= R1.
  - `N = 1`. Result is negative, meaning R0 - R1 < 0, leading to R0 < R1.
- `V = 1`. Overflow, implying (1) the comparison result is wrong and (2) R0 and R1 cannot have the same value (so, we have `Z = 0`). Two situations as well:
  - `N = 0`. Result is mistakenly non negative ==> R0 - R1 < 0, leading to R0 < R1.
  - `N = 1`. Result is mistakenly negative, ==> R0 - R1 > 0, leading to R0 > R1.

The above results can be simplified to the following table:

	N = 0	N = 1
V = 0	Case (1): R0 >= R1	Case (2): R0 < R1
V = 1	Case (3): R0 < R1	Case (4): R0 > R1

Then, we can relate the CCS to the specific cases as follows:

CCS	Cases and conditions	Logic expression
LT	Case (2) OR Case (3)	(V != N)
LE	Case (2) OR Case (3) OR (Z==1)	(V != N) || (Z == 1)
GE	Case (4) OR Case (1)	(V == N)
GT	Case (4) OR (Case (1) AND Z==0)	(V == N) && (Z == 0)

19.3.6 Relationship testing and condition flags for unsigned integers

We skip the derivation part, which is similar to that of the signed numbers and can be found in the Collection, and show the result:

CCS	Cases and conditions	Logic expression
LO	Case (1) OR Case (2)	(C == 0)
LS	Case (1) OR Case (4)	(C == Z)
HS	Case (3) OR Case (4)	(C == 1)
HI	Case (3) OR Case (2)	(C != Z)

19.4 Hand calculation examples

Example 1. Consider **Example 2** of Lctr 18, which is duplicated below:

> **Example 2**. Consider the following operations on signed numbers on a 5-bit system: (a) 5 + 11, (b) (-15) + 11, (c) (-3) - 13, and (d) (-15) - 5. (Note that all the numbers given here can be seen as signed numbers; we need to do the necessary conversions.) Perform the above (a) to (d) operations using the decimal-based approach to determine (1) the final results of the operations in decimal when seeing the operands as both unsigned and signed and (2) the N, Z, C, and V flags.

The solution for Parts (c) and (d) are as follows:

Part (c) (-3) - 13:

- Signed subtraction result: -16
- Unsigned subtraction result: 16
- Status flags: N = 1, Z = 0, C = 1, and V = 0.

Part (d) (-15) - 5:

- Signed subtraction result: 12
- Unsigned subtraction result: 12
- Status flags: N = 0, Z = 0, C = 1, and V = 1.

Verify the following CCSs for signed numbers:

```
a. LT <==> (V != N)
b. LE <==> (V != N) || (Z == 1)
c. GE <==> (V == N)
d. GT <==> (V == N) && (Z == 0)
```

and the following CCSs for unsigned numbers

```
a. LO <==> (C == 0)
b. LS <==> (C == Z)
c. HS <==> (C == 1)
d. HI <==> (C != Z)
```

Solution:

Part (c), signed: -3 < 13:

```
a. LT <==> (V != N): V=0, N=1 ==> (V!=N) is
    ==> LT is
c. LE <==> (V != N) || (Z == 1): (V!=N) is
    ==> LE is
b. GE <==> (V == N): V=0, N=1 ==> (V==N) is
    ==> GE is
d. GT <==> (V == N) && (Z == 0): (V == N) is
    ==> GT is
```

Part (c), unsigned: -3 is the same as 29, which is higher than 13:

```
a. LO <==> (C == 0): C=1 ==> (C==0) is
    ==> LO is
b. LS <==> (C == Z): C=1, Z=0 ==> (C==Z) is
    ==> LS is
c. HS <==> (C == 1): C=1 ==> (C==1) is
    ==> HS is
d. HI <==> (C != Z): C=1, Z=0 ==> (C!=Z) is
    ==> HI is
```

Part (d), signed: -15 < 5:

```
a. LT <==> (V != N): V=1, N=0 ==> (V!=N) is
    ==> LT is
c. LE <==> (V != N) || (Z == 1): (V!=N) is
    ==> LE is
b. GE <==> (V == N): V=1, N=0 ==> (V==N) is
    ==> GE is
d. GT <==> (V == N) && (Z == 0): (V == N) is
    ==> GT is
```

Part (d), unsigned: -15 is the same as 17, which is higher than 5:

```
a. LO <==> (C == 0): C=1 ==> (C==0) is
    ==> LO is
b. LS <==> (C == Z): C=1, Z=0 ==> (C==Z) is
    ==> LS is
c. HS <==> (C == 1): C=1 ==> (C==1) is
    ==> HS is
```

```
d. HI <==> (C != Z): C=1, Z=0 ==> (C!=Z) is
     ==> HI is
```

-:-

19.5 Branch and conditional branch instructions

19.5.1 The branch instruction

The branch instruction has the following syntax:

```
     B     to_label
```

This instruction is used to load the value represented by `to_label`, the address of the instruction we want to branch to, via the PC (program counter) to change the flow of the program.

19.5.2 The most used conditional branch instructions

Adding CCSs to the branch instruction, we obtain the following most used conditional branch instructions.

Relational test	Signed instructions	Unsigned instructions
==	BEQ	BEQ
!=	BNE	BNE
>	BGT	BHI
>=	BGE	BHS
<=	BLE	BLS
<	BLT	BLO

19.6 Implementing basic `if-else` construct in ASM

Now, let's consider how to implement the basic `if-else` construct in ASM based on the implementations in C.

Demo code section 1. Implement the following ternary operator `result = x > y ? a : b;` in both C and ASM functions (using comparison operations or instructions) for two situations: (a) all signed numbers and (b) all unsigned numbers.

Solution:

Part 1. C implementation for signed integers

```c
int mp_x_cmp_y_ab_i_c(int a, int b, int x, int y) {
    if (x > y) {
        return a;
    } else {
        return b;
    }
}
```

Part 2. C implementation for unsigned integers

```c
uint32_t mp_x_cmp_y_ab_u_c(uint32_t a, uint32_t b, uint32_t x,
    uint32_t y) {
    if (x > y) {
        return a;
    } else {
        return b;
    }
}
```

Note that the above two functions are exactly the same except the data types. This is the beauty of C as we can use consistent operators.

Part 3. ASM implementation for signed integers using **negative comparison logic**

```
@ int mp_x_cmp_y_ab_i_c(int a, int b, int x, int y) {
@     if (x > y) {
@         return a;
@     } else {
@         return b;
@     }
@ }
@ int mp_x_cmp_y_ab_i_s(int a, int b, int x, int y)
@   Input:
@     * a -> r0
@     * b -> r1
@     * x -> r2
@     * y -> r3
@   Output:
@     * r0
mp_x_cmp_y_ab_i_s:
    cmp     r2, r3
    ble     mp_x_cmp_y_ab_i_s_else  @ check LE, not GT---
       negative logic

mp_x_cmp_y_ab_i_s_then:
    b       mp_x_cmp_y_ab_i_s_end_if

mp_x_cmp_y_ab_i_s_else:
    mov     r0, r1

mp_x_cmp_y_ab_i_s_end_if:
    bx      lr
```

Note that we can use the *positive comparison logic*, GT; for this case, we will have the *then* code block under the *else* code block, as will be shown in later lectures.

Part 4. ASM implementation for unsigned integers using *negative comparison logic*

```
@ uint32_t mp_x_cmp_y_ab_u_s(uint32_t a, uint32_t b, uint32_t x, uint32_t y) {
@     if (x > y) {
@         return a;
@     } else {
@         return b;
@     }
@ }
@   Input:
@     * a -> r0
@     * b -> r1
@     * x -> r2
@     * y -> r3
@   Output:
@     * r0
mp_x_cmp_y_ab_u_s:
    cmp     r2, r3
    bls     mp_x_cmp_y_ab_u_s_else  @ check LS, not HI---negative logic

mp_x_cmp_y_ab_u_s_then:
    b       mp_x_cmp_y_ab_u_s_end_if

mp_x_cmp_y_ab_u_s_else:
    mov     r0, r1

mp_x_cmp_y_ab_u_s_end_if:
    bx      lr
```

Note that the only difference between Part 3 and Part 4 is that we use conditional branch instructions for signed and unsigned numbers, respectively.

> -:-

See the demo project for the unit test functions of the above functions.

19.7 Implementing basic `for` and `while` constructs in ASM

Now, let's consider how to implement the basic `for` or `while` loops in ASM. We will tweak the `for` loop so that it can be the same as the `while` loop, which will then be implemented by ASM.

Demo code section 2. Consider the calculation of the sum of integers in a given range from start integer s to end integer e, according to the following equation.

$$s = \begin{cases} 0, & \text{if } s > e \\ \sum_{i=s}^{e} i, & \text{otherwise} \end{cases}$$

Implement the above calculation using the following:

- The standard `for` loop in C.
- The standard `while` loop in C.
- A modified `for` loop in C that is essentially the same as the `while` loop.
- A simple implementation of the `for-while` loop in ASM.

Note that we can use very simple equations to do the calculation, but we just want to use the common loops as we need to illustrate the programming of them in ASM.

Solution:

Part 1. Implementation of the range sum with a standard `for` loop in C.

```c
int mp_range_sum_standard_for_c(int s, int e) {
    int32_t sum = 0;
    for (int i = s; i <= e; i++) {
        sum += i;
    }
    return sum;
}
```

Part 2. Implementation of the range sum with a standard `while` loop in C.

```
int mp_range_sum_standard_while_c(int s, int e) {
    int32_t sum = 0;
    int i = s;
    while (i <= e) {
        sum += i;
        i++;
    }
    return sum;
}
```

Part 3. Implementation of the range sum with a modified `for` loop in C which is the **same** as the standard `while` loop.

```
int mp_range_sum_modified_for_c(int s, int e) {
    int32_t sum = 0;
    int i = s;
    for (; i <= e; ) {
        sum += i;
        i++;
    }
    return sum;
}
```

We see that the only difference between this version of the `for` loop and the `while` loop is the `for` and `while` statements. As we will see below, they are both implemented in the same way in ASM.

Part 4. Implementation of the range sum in ASM based on the a modified `for` loop in C or the standard `while` loop in C.

```
@ int mp_range_sum_standard_while_c(int s, int e) {
@     int32_t sum = 0;
@     int i = s;
@     while (i <= e) {
@         sum += i;
@         i++;
@     }
@     return sum;
@ }
```

```
@ int mp_range_sum_for_while_s(int s, int e)
@   Input:
@     * s -> r0
@     * e -> r1
@   Output:
@     * r0
@   Others:
@     * sum -> r2, i -> r3
mp_range_sum_for_while_s:
    ldr     r2, =0  @ sum = 0
    mov     r3, r0  @ r0 can also be used as i directly

mp_range_sum_for_while_s_loop:
    cmp     r3, r1                  @ Part 1 of (i <= e)?
    bgt     mp_range_sum_for_while_s_end_loop   @ Part 2
    add     r2, r2, r3          @ sum += i
    add     r3, #1              @ i++
    b       mp_range_sum_for_while_s_loop

mp_range_sum_for_while_s_end_loop:
    mov     r0, r2              @ return sum
    bx      lr                  @ return sum
```